Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Web - Projet
Gestion des Ventes et des Stocks d'un Magasin
3e année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Réalisé par :	HATHOUTI Mohammed Taha
	JIDAL Ilyas
	KABORÉ Mohammad Sharif Jonathan
Filière :	Cybersécurité
Année :	2025/2026
Enseignants :	M.NOUFEL & M.NAJIB
Date :	1er avril 2026

Table des matières

1 Introduction **3**
- 1.1 Contexte . . . 3
- 1.2 Rappel de la consigne . . . 3
- 1.3 Objectifs du projet . . . 3

2 Choix de conception **4**
- 2.1 Architecture MVC . . . 4
- 2.2 Système de rôles et contrôle d'accès . . . 4
- 2.3 Gestion des stocks avec seuil d'alerte . . . 4
- 2.4 Snapshot de prix dans les lignes de vente . . . 5
- 2.5 Lien fournisseur-utilisateur . . . 5

3 Conception **5**
- 3.1 Diagramme de classes . . . 5
- 3.2 Diagramme de cas d'utilisation . . . 6

4 Implémentation **6**
- 4.1 Vue d'ensemble . . . 6
- 4.2 Schéma de base de données . . . 7
- 4.3 Fonctionnalités clés . . . 7
  - 4.3.1 Authentification et routage par rôle . . . 7
  - 4.3.2 Interface de caisse avec panier JavaScript . . . 7
  - 4.3.3 Recherche avancée de produits . . . 7
  - 4.3.4 Alertes stock et dashboard . . . 8

5 Guide utilisateur **8**
- 5.1 Installation . . . 8
- 5.2 Comptes de test . . . 8
- 5.3 Profil Administrateur . . . 8
- 5.4 Profil Vendeur . . . 8
- 5.5 Profil Fournisseur . . . 9

6 Notre surplus **9**
- 6.1 Architecture avancée . . . 9
- 6.2 Fonctionnalités métier . . . 9
- 6.3 Interface utilisateur . . . 9
- 6.4 Synthèse . . . 10

7 Résultats et analyse **10**
7.1 Points forts validés 10
7.2 Limites identifiées 10

8 Discussion critique **10**
8.1 Points forts 10
8.2 Axes d'amélioration 10

9 Conclusion **11**
9.1 Bilan 11
9.2 Compétences acquises 11
9.3 Perspectives 11

1 Introduction

1.1 Contexte

La gestion des stocks et des ventes constitue un enjeu stratégique pour toute entreprise commerciale. Face à la multiplicité des produits, des fournisseurs et des transactions quotidiennes, les gestionnaires de magasin nécessitent des outils informatiques fiables pour optimiser leurs opérations et prévenir les ruptures de stock.

$$\text{Performance}_{\text{magasin}} = f(\text{Stock}, \text{Ventes}, \text{Traçabilité})$$

Une application web centralisée permet d'améliorer la réactivité opérationnelle, de réduire les pertes liées aux ruptures de stock, et d'assurer une traçabilité complète des transactions commerciales.

1.2 Rappel de la consigne

Le projet consiste à concevoir et développer une application web permettant aux gestionnaires de magasin de suivre et d'optimiser leurs stocks ainsi que leurs ventes.

Fonctionnalités requises :

1. Système d'authentification sécurisé avec gestion de rôles multiples
2. Gestion complète du catalogue produits (CRUD) avec catégories et fournisseurs
3. Suivi des stocks en temps réel avec alertes de seuil
4. Interface de vente (caisse) avec constitution de panier et calcul automatique
5. Historique des ventes et tableau de bord statistique

Rôles utilisateurs :

— Administrateur : accès total, CRUD utilisateurs, vue d'ensemble du chiffre d'affaires
— Vendeur : accès à la caisse, au catalogue et à son historique de ventes
— Fournisseur (optionnel) : visualisation des stocks de ses produits

Technologies imposées :

— Langage : PHP, paradigme 100% Orienté Objet
— Base de données : MySQL avec PDO et requêtes préparées obligatoires
— Architecture : séparation stricte logique / vue, type MVC recommandé
— Frontend : HTML, CSS, JavaScript
— Sécurité : mots de passe hachés

1.3 Objectifs du projet

Les objectifs principaux sont la maîtrise de la programmation orientée objet en PHP, la conception d'une architecture MVC robuste et sécurisée, et le développement d'un système fonctionnel répondant à un besoin métier réel.

$$\text{Compétences} = \text{POO PHP} + \text{MVC} + \text{PDO} + \text{Sécurité} + \text{Travail d'équipe}$$

2 Choix de conception

2.1 Architecture MVC

Notre projet suit rigoureusement le pattern Modèle-Vue-Contrôleur, séparant les responsabilités en trois couches distinctes avec un front controller unique.

$$\text{Application} = \text{Modèle} \oplus \text{Vue} \oplus \text{Contrôleur}$$

Le **Modèle** regroupe les classes métier (User, Product, Category, Supplier, Sale, SaleItem) contenant la logique d'accès aux données via PDO. La **Vue** gère uniquement l'affichage HTML sans aucune logique métier. Le **Contrôleur** orchestre les interactions entre modèles et vues, valide les données et gère les redirections.

Avantage	**Explication**
Séparation des préoccupations	Chaque couche a une responsabilité unique
Sécurité centralisée	Gardes de session et rôles dans Auth.php
Maintenabilité	Modifications localisées sans effet de bord
Front controller unique	Un seul point d'entrée (index.php)
Travail d'équipe	Développement parallèle sur différentes couches

Table 1 – Avantages du pattern MVC adopté

2.2 Système de rôles et contrôle d'accès

Nous avons implémenté une classe statique `Auth` centralisant toute la logique d'authentification et de contrôle d'accès par rôle.

$$\text{Rôles} \in \{\text{admin}, \text{vendeur}, \text{fournisseur}\}$$

Les transitions d'accès sont contrôlées par des méthodes de garde :

$$\text{Requête} \xrightarrow{\text{requireLogin()}} \text{Session active} \xrightarrow{\text{requireRole()}} \text{Accès autorisé}$$

Ce système garantit qu'aucune page protégée n'est accessible sans authentification valide, et qu'aucune action d'écriture n'est exécutable sans le rôle approprié.

2.3 Gestion des stocks avec seuil d'alerte

Chaque produit dispose d'un `seuil_alerte` personnalisé. La méthode métier `isLowStock()` compare le stock actuel au seuil et retourne un booléen utilisé pour l'affichage visuel.

$$\text{isLowStock}() = (\text{quantite_stock} \leq \text{seuil_alerte})$$

La décrémentation du stock lors d'une vente est atomique — effectuée directement en SQL pour éviter les conditions de course :

$$\text{quantite_stock} \leftarrow \text{quantite_stock} - \text{quantite_vendue}$$

2.4 Snapshot de prix dans les lignes de vente

Le champ `prix_unitaire` de la table `sale_items` capture le prix de vente au moment de la transaction, indépendamment des modifications futures du catalogue.

$$\text{total_ttc} = \sum_{i=1}^{n} \text{prix_unitaire}_i \times \text{quantite}_i$$

Cette approche garantit l'intégrité de l'historique commercial même si les prix évoluent.

2.5 Lien fournisseur-utilisateur

La table `suppliers` dispose d'un champ `id_user` optionnel liant un fournisseur à un compte utilisateur. Lorsqu'un utilisateur avec le rôle `fournisseur` se connecte, le système retrouve automatiquement son fournisseur associé et filtre l'affichage.

$$\text{Fournisseur connecté} \xrightarrow{\text{findByUserId()}} \text{Ses produits uniquement}$$

3 Conception

3.1 Diagramme de classes

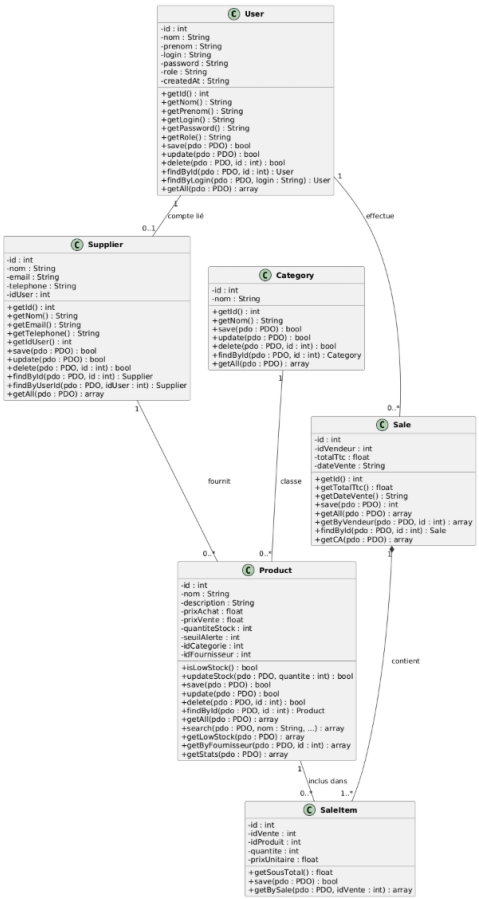

FIGURE 1 – Diagramme de classes — Package models avec relations

3.2 Diagramme de cas d'utilisation

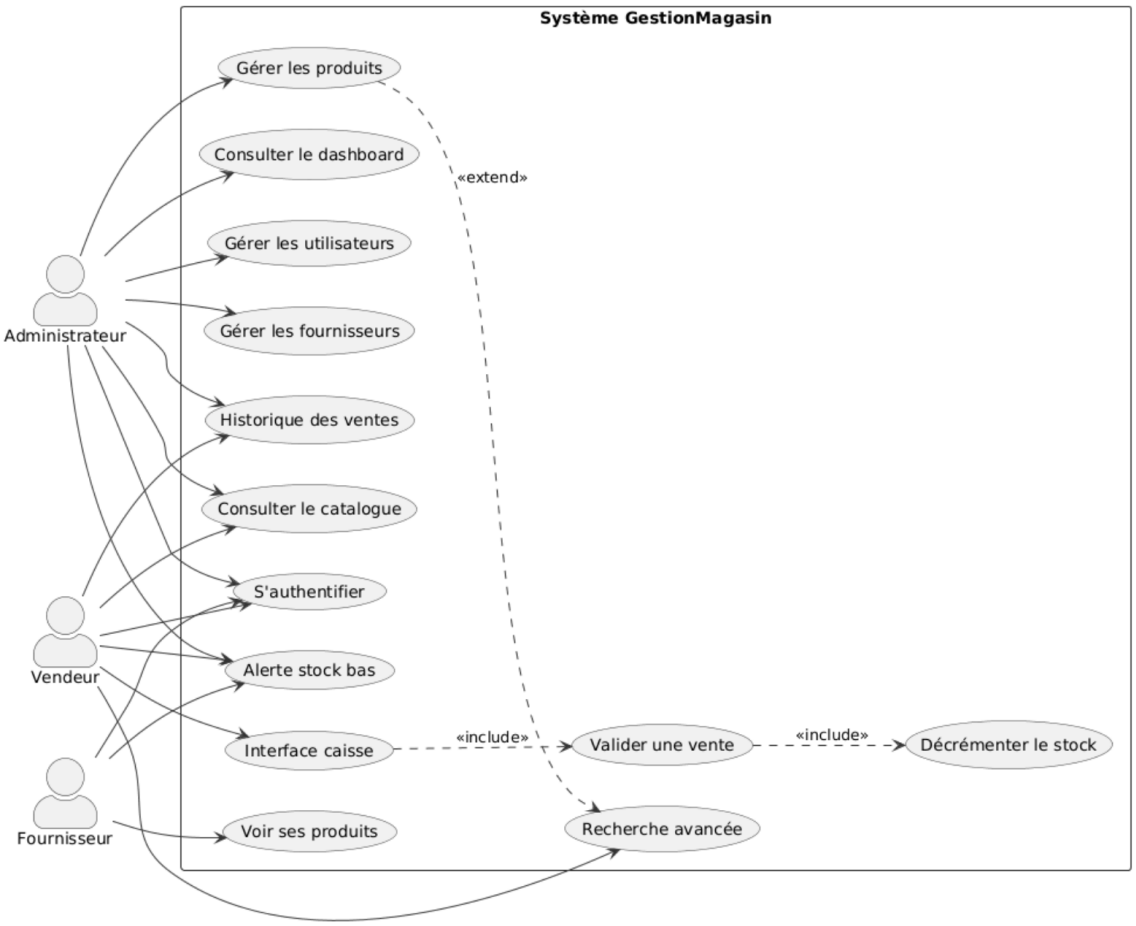

FIGURE 2 – Diagramme de cas d'utilisation du système

4 Implémentation

4.1 Vue d'ensemble

Le projet comprend 15 fichiers PHP organisés en 4 couches MVC, 1 base de données MySQL avec 6 tables, 3 rôles utilisateurs distincts, et 12 vues complètes.

Couche	Fichiers	Rôle
models/	6	Entités métier + accès PDO
controllers/	4	Logique applicative + validation
views/	12	Interfaces HTML/PHP
src/	2	Noyau (Auth, Controller abstraite)

TABLE 2 – Organisation du projet

4.2 Schéma de base de données

La base de données suit une architecture relationnelle normalisée avec 6 tables interconnectées par des clés étrangères.

Structure des tables :

- Table `users` : id, nom, prénom, login, password (bcrypt), role (ENUM), created_at
- Table `categories` : id, nom
- Table `suppliers` : id, nom, email, téléphone, id_user (FK optionnel vers users)
- Table `products` : id, nom, description, prix_achat, prix_vente, quantite_stock, seuil_alerte, id_categorie (FK), id_fournisseur (FK)
- Table `sales` : id, id_vendeur (FK), total_ttc, date_vente
- Table `sale_items` : id, id_vente (FK), id_produit (FK), quantite, prix_unitaire (snapshot)

Contraintes d'intégrité :

- `ON DELETE CASCADE` : suppression d'une vente entraîne la suppression de ses lignes
- `ON DELETE SET NULL` : suppression d'une catégorie ou d'un fournisseur ne supprime pas les produits associés
- `ON DELETE SET NULL` : suppression d'un compte utilisateur ne supprime pas le fournisseur associé

4.3 Fonctionnalités clés

4.3.1 Authentification et routage par rôle

La classe statique `Auth` vérifie les identifiants via `password_verify()` contre le hash bcrypt stocké. En cas de succès, le login, rôle et id sont stockés en session. Un cookie `derniere_connexion` est créé avec une durée de vie de 30 jours. Le front controller `index.php` dispatche chaque requête vers le contrôleur approprié selon le paramètre `$_GET['page']`.

4.3.2 Interface de caisse avec panier JavaScript

La caisse est entièrement dynamique côté client. Un objet JavaScript `panier` est mis à jour à chaque modification de quantité, recalcule le total TTC en temps réel, et génère les champs `hidden` soumis au serveur. La validation côté serveur vérifie ensuite la disponibilité en stock avant d'enregistrer la vente.

4.3.3 Recherche avancée de produits

La méthode `Product::search()` construit dynamiquement une requête SQL avec des clauses `WHERE` conditionnelles selon les filtres actifs (nom, catégorie, fournisseur). L'utilisation de `WHERE 1=1` permet d'enchaîner les conditions sans cas particulier pour la première clause.

4.3.4 Alertes stock et dashboard

La méthode `getLowStock()` retourne tous les produits dont le stock est inférieur ou égal au seuil d'alerte. Le dashboard admin agrège en une seule requête SQL le chiffre d'affaires total, le nombre de ventes et le panier moyen, ainsi que la valeur totale du stock et le nombre d'alertes actives.

5 Guide utilisateur

5.1 Installation

Pré-requis : PHP 8.0+, MySQL Server 5.7+, serveur web (Apache ou serveur built-in PHP)

Configuration base de données :

1. Créer la base : `CREATE DATABASE GestionMagasin`
2. Créer l'utilisateur : `CREATE USER 'magasin'@'localhost' IDENTIFIED BY 'magasin123'`
3. Accorder les privilèges : `GRANT ALL ON GestionMagasin.* TO 'magasin'@'localhost'`
4. Exécuter le script `database.sql` fourni
5. Lancer `setup.php` dans le navigateur, copier les `UPDATE` générés et les exécuter
6. Supprimer `setup.php`

Exécution :

```
php -S localhost:8080
```

5.2 Comptes de test

Login	Mot de passe	Rôle	Accès
admin	admin123	Administrateur	Complet
vendeur1	vendeur123	Vendeur	Caisse + Catalogue
fournisseur1	fourn123	Fournisseur	Ses produits uniquement

Table 3 – Comptes utilisateurs de test

5.3 Profil Administrateur

Fonctionnalités : dashboard avec chiffre d'affaires global, CRUD complet sur les produits, catégories, fournisseurs et utilisateurs, alertes stock, historique de toutes les ventes.

5.4 Profil Vendeur

Fonctionnalités : consultation du catalogue avec recherche avancée, interface caisse avec panier dynamique et calcul TTC automatique, historique de ses propres ventes.

5.5 Profil Fournisseur

Fonctionnalités : consultation des produits qu'il fournit en lecture seule, visualisation des alertes stock sur ses propres produits.

6 Notre surplus

Au-delà des exigences minimales du cahier des charges, nous avons implémenté plusieurs fonctionnalités supplémentaires.

6.1 Architecture avancée

Front controller unique : Le sujet recommandait une structure MVC. Nous avons implémenté un front controller complet avec une classe `Controller` abstraite, un routing par switch sur `$_GET['page']`, et une méthode `render()` avec `extract()` pour passer les données aux vues.

Classe Auth statique : Centralisation de toute la logique d'accès avec cinq méthodes (`requireLogin()`, `requireRole()`, `isAdmin()`, `isVendeur()`, `isFournisseur()`) — au lieu des gardes de session inline.

Hydratation avec JOIN : Les requêtes `getAll()` et `findById()` de `Product` effectuent des `LEFT JOIN` sur `categories` et `suppliers`, évitant les requêtes N+1 en vue.

6.2 Fonctionnalités métier

Rôle fournisseur complet : Implémentation du rôle optionnel avec lien `id_user` dans `suppliers`, filtrage automatique des produits et alertes personnalisées.

Seuil d'alerte par produit : Chaque produit dispose de son propre `seuil_alerte` configurable, contrairement à une valeur globale.

Snapshot de prix : Préservation du prix unitaire au moment de la vente dans `sale_items`, garantissant l'intégrité de l'historique.

Recherche multi-critères : Filtrage dynamique combinant nom, catégorie et fournisseur avec construction SQL conditionnelle.

6.3 Interface utilisateur

Sidebar de navigation : Navigation latérale adaptative selon le rôle — les sections Administration sont masquées pour les vendeurs et fournisseurs.

Caisse JavaScript : Panier entièrement dynamique sans rechargement de page, avec calcul TTC en temps réel, mise à jour visuelle instantanée et validation de stock côté client et serveur.

Indicateurs visuels stock : Badges colorés (vert/rouge) sur chaque ligne produit selon `isLowStock()`, avec surlignage des lignes critiques dans la caisse.

6.4 Synthèse

Fonctionnalité	Demandé ?	Implémenté
Rôle Fournisseur	Optionnel	Complet avec filtrage
Front controller MVC	Recommandé	Architecture complète
Classe Auth statique	Non	5 méthodes de garde
Panier JavaScript dynamique	Non	Temps réel, sans rechargement
Snapshot prix dans ventes	Non	Intégrité historique garantie
Recherche multi-critères	Non	3 filtres combinables
JOIN en hydratation	Non	Évite les requêtes N+1
Sidebar adaptative par rôle	Non	Masquage conditionnel

Table 4 – Comparaison cahier des charges vs implémentation

7 Résultats et analyse

7.1 Points forts validés

Sécurité (requêtes préparées PDO, mots de passe bcrypt, gardes de session systématiques), cohérence des données (contraintes FK, snapshot de prix, décrémentation atomique du stock), ergonomie (navigation sidebar, alertes visuelles, panier dynamique).

7.2 Limites identifiées

Absence de pagination sur les listes de produits volumineuses, pas de gestion des retours de marchandise, historique des modifications de stock non tracé, absence de système de catégories hiérarchiques.

8 Discussion critique

8.1 Points forts

Architecture MVC stricte : Séparation nette des responsabilités avec classe abstraite `Controller`, facilitant l'ajout de nouvelles fonctionnalités sans modifier l'existant.

Sécurité multicouche : Validation côté serveur systématique, requêtes préparées sur toutes les interactions base de données, hachage bcrypt des mots de passe, et contrôle d'accès par rôle sur chaque action.

Expérience utilisateur : Interface caisse fluide grâce au JavaScript, indicateurs visuels de stock, navigation adaptée au rôle.

8.2 Axes d'amélioration

Pagination : Implémenter une pagination SQL (`LIMIT` / `OFFSET`) pour les catalogues volumineux.

Traçabilité des stocks : Ajouter une table `stock_movements` enregistrant chaque entrée/sortie avec date, quantité et motif.

Transactions SQL : Encapsuler la validation de vente (insertion `sales` + `sale_items` + mise à jour stock) dans une transaction `BEGIN` / `COMMIT` pour garantir l'atomicité en cas d'erreur.

9 Conclusion

9.1 Bilan

Ce projet nous a permis de mettre en pratique l'ensemble des concepts de programmation web orientée objet étudiés en cours, tout en réalisant une application complète et fonctionnelle répondant à un besoin métier réel.

Nous avons maîtrisé la POO PHP (classes, héritage, méthodes statiques), implémenté le pattern MVC avec front controller, sécurisé les accès avec PDO et requêtes préparées, géré les sessions et cookies, et dépassé le cahier des charges avec plusieurs fonctionnalités supplémentaires.

9.2 Compétences acquises

Techniques : PHP orienté objet, PDO et MySQL, architecture MVC, JavaScript dynamique, gestion de sessions/cookies, sécurité web.

Méthodologiques : Analyse des besoins, conception de base de données relationnelle, travail en équipe, gestion de version Git.

9.3 Perspectives

Ce projet constitue une base solide pouvant évoluer vers une API REST (JSON) consommée par une application mobile, un système de codes-barres pour la gestion physique du stock, ou une intégration avec des passerelles de paiement pour la caisse.

Projet actuel $\rightarrow$ Évolutions futures multiples

Références

— **Cours :** M.NOUFEL & M.NAJIB, *Programmation Web - Projet*, 3A Cybersécurité, UIR - ESIN 2025-2026

— **Documentation PHP :** *PHP Manual*, https://www.php.net/manual/fr/

— **Documentation PDO :** *PHP Data Objects*, https://www.php.net/manual/fr/book.pdo.php

— **Design Patterns :** Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley.

— **Assistant IA :** Claude (Sonnet 4.6), Anthropic, https://claude.ai — assistance au développement, debugging et documentation